

PRAKTIKUM INFORMATIK 2

M. Niemetz

Übung: Objektorientiertes Design

Objektorientierter Sudoku-Löser

Mai 2025, Version: 1.41

1 Einleitung

Diese Aufgabe entstand ursprünglich aus einer Idee der Vorlesungsteilnehmer im Sommer 2017 (Studiengang REE).

Ihrem inzwischen erreichten Ausbildungsstand entsprechend, bietet diese Aufgabe kein vorgegebenes objektorientiertes Lösungsdesign, sondern soll Ihnen die Möglichkeit geben, Ihr Können zu vertiefen indem Sie selbst einen objektorientierten Lösungsansatz entwickeln und diesen umsetzen.

Dies erleichtert das Verständnis fremder Lösungsansätze und versetzt Sie auch in die Lage, Probleme aus dem Ingenieursalltag in entsprechende Softwarelösungen umzusetzen.

In dieser Aufgabe soll ein Sudoku-Löser mit einem objektorientierten Ansatz entwickelt werden.

Um den beabsichtigten Lernerfolg zu erzielen, sollten Sie natürlich die Lösung selbst erarbeiten und dabei auch unbedingt darauf achten, dass mehrere Objekte verschiedener Klassen bei der Lösung zusammenarbeiten.

Vorbereitung

☐ ☕ **Sudoku-Regeln:** Informieren Sie sich über die Sudoku-Regeln. 2

2 Vorbereiten des Projekts

Legen Sie ein neues leeres Codeblocks-Projekt in Ihrem Git-Projektverzeichnis auf Ihrem Arbeits-PC an.

Fügen Sie alle Dateien zu Ihrem git-Projekt hinzu.



Erzeugen Sie eine neue Version und laden Sie sie auf den GitLab-Server hoch.



Bereiten Sie das Projekt für die Entwicklung vor.

3 Design einer Lösung

Sollten Sie die Regeln von Sudoku nicht kennen, informieren Sie sich!

Die grundsätzlich einfachste Idee, ein Sudoku-Rätsel zu lösen, ist, stumpsinnig alle Möglichkeiten, die Zahlen in die Felder zu setzen, durchzuprobieren.

Recherche ☕

Sudoku-Regeln

Damit der Rechenaufwand für dieses „Herumprobieren“ nicht ins Unermessliche wächst, ist es sinnvoll, das Sudoku-Spielfeld schrittweise nacheinander zu füllen und dabei auftretende „Fehlzüge“ sofort als solche zu erkennen, und wieder zurückzunehmen statt mit dem falsch gefüllten Feld mit dem Füllen des Spielfeldes weiter fortzufahren.

Hiermit zerfällt der Algorithmus in zwei wesentliche Schritte:

- **Erstellen einer neuen Teillösung:** Hierfür müssen Sie ein zusätzliches Feld mit einer Zahl belegen, oder eine als ungünstig ermittelte Zahl durch eine andere ersetzen. Um dabei alle Varianten durchzuprobieren, empfiehlt sich ein rekursives Vorgehen, das prinzipiell alle Kombinationen abdeckt, aber „tote Pfade“ abbricht, statt sie weiter zu erproben.
- **Prüfen der Teillösung auf Gültigkeit:** Die Teillösungen (also teilweise gefüllte Sudoku-Spielpläne) müssen geprüft werden, ob sie den Sudoku-Regeln genügen oder ob es zu Verstößen kommt.

Überlegen Sie sich zunächst:

- Wie sollen die Zahlenfelder im Sudoku abgespeichert werden? Benötigen Sie nur einfache Zahlen oder müssen auch weitere Informationen gespeichert werden?
- Welche Klassen mit welchen Datenelementen und Funktionen benötigen Sie, um die verschiedenen Sudoku-Regeln gut abbilden und prüfen zu können?
- In welchen Klassen sollen welche Funktionen platziert werden?
- Welche Objekte müssen mit welchen anderen Objekten zusammenarbeiten? Wie kommt die Kommunikation zustande?

Bei dieser Entwicklungsarbeit helfen Ihnen Fragen wie „Welche Daten werden gebraucht?“ und „Was muss alles getan werden?“, wobei die wichtigste Frage ist „Wer (welches Objekt welcher Klasse) sollte diese Aufgabe jeweils erledigen?“.

Unter diesen Gesichtspunkten sollten Sie die Aufgaben und Daten auf Objekte verschiedener Klassen verteilen.

Wichtig: Bemühen Sie sich um einen guten **objektorientierten Ansatz**, in dem mehrere Klassen zusammenwirken um das Problem zu lösen. Achten Sie auf gute Kapselung von Daten und zugehörigen Methoden! Andernfalls reduzieren Sie den Lernerfolg!

Hinweis: Häufig ist es eine gute Idee, solche Entwicklungen zunächst auf Papier (oder elektronischem Äquivalent) durchzuführen und sich nicht mit in der Regel umständlichen elektronischen Modellierungswerkzeugen zu belasten.



Entwerfen Sie einen objektorientierten Lösungsansatz und dokumentieren Sie ihn in einem möglichst kompletten UML-Modell.

Hinweis: Auch wenn die verketteten Listen in diesem Semester eine große Rolle gespielt haben, sind sie nicht bei jedem Problem unbedingt die beste Lösung oder gar nötig.

4 Der Lösungsalgorithmus

Überlegen Sie sich zunächst im Detail, wie Ihr Lösungsalgorithmus im Detail ablaufen soll.

Hinweis: Die im Folgenden zu findenden Hinweise reichen nicht aus, um mit dem Kodieren einer Lösung zu beginnen.

Sie müssen sich zuerst die Lösung für viele kleine und große Herausforderungen überlegen, bevor Sie mit der Programmierung beginnen.

Sonst besteht die sehr große Gefahr, dass das Vorhaben in großem Frust endet.

Für die Lösung des Sudokus ist es wie oben bereits angedeutet am einfachsten, wenn Sie einfach einen primitiven Ansatz wählen, in dem rekursiv alle noch möglichen Ziffern eingesetzt werden. Dabei wird in jedem Schritt das Ergebnis auf Gültigkeit geprüft und abhängig vom Ergebnis der Prüfung weiter eingesetzt oder die zuletzt gesetzte Ziffer entfernt und durch eine andere ersetzt (sofern noch Optionen zur Verfügung stehen).

Damit sieht eine Funktion `sudokuschritt`, die diesen Rekursionsschritt ausführt, in etwa folgendermaßen aus:

- Setze nacheinander die Zahlen 1 bis 9 in das aktuelle Feld ein.
- Prüfe, ob das Ergebnis noch eine gültige Sudoku-Teillösung ist. Falls nein: Versuche nächste einzusetzende Zahl; Falls ja: Erstmal weiter:
- Rufe `sudokuschritt` für das nächste freie Feld (falls verfügbar) auf und prüfe den Rückgabewert. Falls eine gültige Sudoku-Komplettlösung gefunden wurde, beende die Funktion und gib diese Information (z.B. `true`) zurück.
- Andernfalls fahre mit dem weiteren Einsetzen von Ziffern im aktuellen Feld fort, sofern noch nicht alles probiert wurde.
- Wenn im aktuellen Feld nichts mehr zu probieren ist, dann leere das aktuelle Feld und beende die Funktion. Der Rückgabewert soll dann signalisieren, dass keine vollständige Lösung gefunden wurde (also z.B. `false`).

Wenn das Sudoku auf diese Art komplett erfolgreich gefüllt ist, kann die Lösung ausgegeben werden.

Wichtig: Beachten Sie, dass Sie die Felder der ursprünglichen Aufgabenstellung des jeweiligen Sudoku-Rätsels bei diesem Probiervorgang *nicht* verändern dürfen!

Sie brauchen also ein Konzept, solche Felder zu erkennen und beim Probieren zu ignorieren.

Hinweis: Der wichtigste Lerneffekt bei dieser Aufgabe ist die Fähigkeit, ein objektorientiertes Design zu erstellen.

Dieses objektorientierte Design ist insbesondere bei Prüfen der Sudoku-Regeln sehr hilfreich.

Sie sollten also insbesondere nicht zögern, Ihr ursprüngliches Design anzupassen, falls es sich als nicht tragfähig erweist.

5 Die Implementierung

Hinweis: Für den Test ist es hilfreich, Sudokus aus Dateien einlesen zu können.



Implementieren Sie Ihre Lösung und testen Sie sie.

Implementieren Sie Ihre Lösung und testen Sie das Programm an Sudoku-Aufgaben.

Beispiel:

-1	6	-1	4	9	-1	8	3	7
-1	-1	-1	5	6	1	-1	9	-1
2	-1	4	-1	-1	-1	1	-1	5
-1	-1	1	2	-1	3	6	7	-1
6	-1	-1	-1	-1	-1	-1	4	-1
-1	4	-1	8	7	-1	9	5	-1
7	2	-1	6	8	5	4	-1	-1
-1	-1	9	3	-1	7	5	8	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1

Hinweis: Leere Felder sind hier im Beispiel mit "-1" gekennzeichnet.

Hier noch ein schwierigeres Beispiel:

5	-1	-1	-1	6	-1	-1	1	-1
-1	-1	-1	8	-1	-1	7	-1	-1
7	-1	1	-1	-1	9	4	-1	-1
-1	-1	5	-1	-1	-1	-1	-1	4
6	-1	7	9	-1	8	2	-1	3
2	-1	-1	-1	-1	-1	6	-1	-1
-1	-1	4	1	-1	-1	5	-1	9
-1	-1	6	-1	-1	3	-1	-1	-1
-1	5	-1	-1	4	-1	-1	-1	2